

Prompt de la IA

Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran resultados, así como la clase que realiza la carga y la detección del evento para convertir la expression.

Objetivo:

Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase que contenga una pila y un método para llevar acabo la conversion.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces la clase Stack existente. Debes utilizarla e integrarte a su forma actual de trabajo.
3. Respeta la organización actual de archivos, clases, nombres de métodos y estilo de programación del proyecto.
4. Si necesitas proponer cambios, indícalos como cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse.
5. La solución debe estar hecha con programación orientada a objetos.
6. La interfaz gráfica debe seguir usando el diseño generado con Qt Designer; no cambies el archivo .ui salvo que sea estrictamente necesario.
7. La aplicación debe permitir capturar una expresión infija, convertirla a posfija y mostrar el resultado en la interfaz.
8. Se asume que la expression infija capturada es válida ya que de otra forma la conversion puede fallar.
9. Usa operadores básicos: +, -, \*, /, \$(Potencia) y paréntesis.
10. Explica brevemente cómo funciona el algoritmo y cómo se utiliza la pila en la conversion, para esto genera un archivo .md que contenga esta explicación.

Código existente:

[Pega aquí la clase Stack]

[Pega aquí el Código del archivo main.py]

[Pega aquí el Código de la clase que carga la interfaz]

Resultado esperado de tu respuesta:

- Primero analiza la estructura del código existente.
- Después propón la clase con el método para convertir de infija a posfija.
- Luego indica cómo conectar ese método con el botón o evento de la interfaz gráfica.
- Finalmente proporciona pruebas para comprobar que funciona.
- No omitas explicación, porque debo entender el código para defenderlo en clase ante el profesor y mis compañeros.

Resultado

¿Cómo funciona el algoritmo?

Se lee la expresión infija de izquierda a derecha carácter por carácter:

1. **Si es un operando** (número o letra): Se pasa directamente al resultado posfijo.
2. **Si es un paréntesis de apertura** ( : Se hace `push` a la pila.
3. **Si es un paréntesis de cierre** ) : Se van sacando ( `pop` ) elementos de la pila y añadiendo al resultado hasta encontrar el paréntesis de apertura ( , el cual se elimina de la pila pero no se pasa al resultado.
4. **Si es un operador** ( + , - , \* , / , \$ ): Mientras la pila no esté vacía y el operador en el tope de la pila tenga **mayor o igual precedencia** que el operador actual, se hace `pop` de la pila al resultado. Luego, el operador actual se hace `push` a la pila.
5. **Al finalizar la lectura:** Todos los operadores que queden en la pila se sacan ( `pop` ) y se agregan al final del resultado.

2. Propuesta de la nueva Clase de Conversión

+ Pregunta a Gemini Flash 

```
# converter.py
```

```
# Asumiendo que tu clase Stack está en un archivo llamado stack_file.py
```

```
# Modifica el import según el nombre real de tu archivo de la pila.
```

```
from stack_file import Stack
```

```
class ExpressionConverter:
```

```

def __init__(self):

    # La clase CONTIENE una pila, cumpliendo el requerimiento de POO (Composición)

    self.stack = Stack()


    # Definimos las prioridades de los operadores (Precedencia)

    self.precedence = {

        '+': 1,

        '-': 1,

        '*': 2,

        '/': 2,

        '$': 3 # Potencia (mayor prioridad)

    }


def _get_precedence(self, operator):

    """Método auxiliar para obtener la prioridad de un operador."""

    return self.precedence.get(operator, 0)


def infix_to_postfix(self, infix_expression: str) -> str:

    """Método principal que realiza la conversión utilizando la Pila."""

    postfix = []

    # Limpiamos espacios en blanco por si acaso

    infix_expression = infix_expression.replace(" ", "")


    for char in infix_expression:

        # Caso 1: Es un operando (Letra o número)

        if char.isalnum():

            postfix.append(char)


        # Caso 2: Paréntesis de apertura

```

```

elif char == '(':
    self.stack.push(char)

# Caso 3: Paréntesis de cierre
elif char == ')':
    # Sacamos elementos hasta encontrar el '('
    while not self.stack.is_empty() and self.stack.peek() != '(':
        postfix.append(self.stack.pop())
    # Sacamos el '(' de la pila sin guardarlo en el resultado
    if not self.stack.is_empty():
        self.stack.pop()

# Caso 4: Es un operador (+, -, *, /, $)
else:
    # Mientras haya operadores en la pila con mayor o igual prioridad
    while (not self.stack.is_empty() and
           self._get_precedence(self.stack.peek()) >= self._get_precedence(char)):
        postfix.append(self.stack.pop())
    # Insertamos el operador actual en la pila
    self.stack.push(char)

# Al terminar el ciclo, vaciamos lo que quede en la pila
while not self.stack.is_empty():
    postfix.append(self.stack.pop())

# Unimos la lista para retornar el string final
return "".join(postfix)

```

```
# --- PASO 1: Agregar este import al inicio de tu archivo de interfaz ---
```

```
from converter import ExpressionConverter
```

```
# --- DENTRO DE TU CLASE DE LA VENTANA (Ej. class MainWindow...) ---
```

```
def __init__(self):
```

```
    # ... (Tu código existente de setupUi, super, etc.) ...
```

```
# --- PASO 2: Instanciar el convertidor como atributo de la clase ---
```

```
self.converter = ExpressionConverter()
```

```
# --- PASO 3: Conectar el botón al método del evento ---
```

```
# Cambia 'btn_convertir' por el objectName real de tu botón en Qt Designer
```

```
self.ui.btn_convertir.clicked.connect(self.procesar_conversion)
```

```
# --- PASO 4: Agregar este nuevo método en la clase ---
```

```
def procesar_conversion(self):
```

```
    """Manejador del evento del botón para realizar la conversión."""
```

```
    # 1. Recuperar el texto del input (Cambia 'txt_infijo' por tu objectName)
```

```
    expresion_infija = self.ui.txt_infijo.text()
```

```
    # Validación simple por seguridad
```

```
    if not expresion_infija:
```

```
        return
```

```
    # 2. Llamar al método de conversión de la otra clase
```

```
    expresion_posfija = self.converter.infix_to_postfix(expresion_infija)
```

# 3. Mostrar el resultado en la interfaz (Cambia 'txt\_posfijo' por tu objectName)

```
self.ui.txt_posfijo.setText(expresion_posfija)
```

| Expresión Infija<br>(Entrada) | Expresión Posfija (Resultado<br>Esperado) | Explicación para la clase                                                                                       |
|-------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>a+b*c</code>            | <code>abc*+</code>                        | La multiplicación <code>*</code> tiene más prioridad que <code>+</code> , por eso se resuelve antes en posfijo. |
| <code>(a+b)*c</code>          | <code>ab+c*</code>                        | Los paréntesis obligan a la suma <code>+</code> a salir de la pila antes que el <code>*</code> .                |
| <code>a+b-c</code>            | <code>ab+c-</code>                        | Al tener igual prioridad, por asociatividad de izquierda a derecha sale primero el <code>+</code> .             |